Predicting coronary heart disease from a 300 000-respondent health survey: where the accuracy actually comes from

Arthus Duval, Matthias Quin, Alban Breart de Boisanger
CS-433 Machine Learning, EPFL

***Abstract*—We predict coronary heart disease from the 2015 BRFSS telephone survey (328 135 respondents, 321 columns, 8.8 % positives) using only NumPy. Our best model, L_2-regularised logistic regression fitted by Newton's method, reaches an out-of-fold F_1 of 0.441. Our ablation shows that the decision threshold, not the model or the cleaning, carries almost all of the performance: tuning it raises F_1 from 0.259 to 0.441, while our entire hand-crafted decoding of the survey codes is worth less than 0.005. We also find the one-hot design matrix to be rank-deficient, so the normal equations of least squares have no unique solution until redundant columns are removed.**

I. Introduction

The Behavioral Risk Factor Surveillance System (BRFSS) is an annual telephone survey of U.S. residents. Given the 321 answers of a respondent, the task is to predict whether that person has been diagnosed with myocardial infarction or coronary heart disease (MICHD). Two properties of the data drive every decision in this report. First, the classes are strongly imbalanced: only $8.8\,\%$ of the respondents are positive, so accuracy is meaningless (predicting "healthy" for everybody already scores $91.2\,\%$) and the competition is scored with the F_1 score of the positive class. Second, the data is a questionnaire, not a set of measurements: $44.8\,\%$ of the entries are missing, and the numbers stored in the file are mostly category codes.

We built a pipeline that turns the survey codes into meaningful numbers, fits a regularised logistic model, and treats the decision threshold as a hyperparameter. All results below are 5-fold cross-validated, with the preprocessing re-estimated inside every training fold so that no statistic from a validation fold reaches the model.

II. Data and feature engineering

A. What the raw columns look like

Our exploratory analysis (`eda.py`) shows that 220 columns take at most ten distinct values and are categorical codes, 80 are genuinely numerical, and the rest are administrative (identifiers, interview dates, survey design weights). Missingness is very uneven — 147 columns are missing more than half of the time and 99 more than $90\,\%$ — because whole blocks of questions are only asked to a sub-sample of respondents.

B. Cleaning

The survey reserves codes for non-answers (7/9, 77/99, 777/999 depending on the field width) and uses compound codes for quantities: $1xx$ means xx times per week, $2xx$ per month and $3xx$ per year; weight is either in pounds or in kilograms offset by 9000; height is either `ftii` or centimetres offset by 9000. Taken literally, a respondent who refuses to answer a frequency question appears to exercise 999 times a week. We decode the compound codes into a single unit and map the reserved codes to missing values.

Columns with few distinct values are one-hot encoded against their most frequent level; the others are winsorised at the 1st and 99th percentiles, median-imputed and standardised. For every column missing more than $2\,\%$ of the time we add a "was missing" indicator: in a survey a non-answer is itself an answer. Finally we append the square of every numerical feature, so the linear model can represent the U-shaped effect of age or body mass index.

C. A rank-deficient design matrix

One-hot encoding a survey creates exact linear dependencies: a missingness indicator is often already implied by the level dummies of the same column. On our default pipeline the design matrix has 415 columns but $X^\top X$ has rank 344, so `least_squares` raises a singular-matrix error. Either one regularises (any $\lambda > 0$ makes $X^\top X + 2N\lambda I$ invertible) or one removes the redundant columns. We do the latter, with an incremental Cholesky factorisation of the Gram matrix that keeps a column only when it adds a direction the selected ones cannot represent. This restores full rank without changing the span, and lets all six methods run on identical features.

III. Models

A. The six required methods

We implemented the six methods of the project statement in `implementations.py`, solving every linear system with `np.linalg.solve` rather than inverting a matrix. For the iterative methods the step size is not a free choice: gradient descent on the mean squared error converges if and only if $\gamma < 2/\lambda_{\max}(X^\top X/N)$. Measuring $\lambda_{\max} = 170.2$ on our design matrix predicts a stability limit of $\gamma = 0.0117$, which the data confirms exactly: $\gamma = 0.01$ converges to a loss of 0.137 while $\gamma = 0.012$ diverges to 10^{10} within 300 iterations. For stochastic gradient descent the relevant bound involves the curvature of a *single* sample, $2/\max_n \|x_n\|^2 = 1.45 \times 10^{-4}$, two orders of magnitude smaller, which is why the two methods appear with very different step sizes in Table I.

TABLE I
THE SIX REQUIRED METHODS AND OUR VARIANT, 5-FOLD CROSS-VALIDATION ON THE SAME FEATURES, WITH THE DECISION THRESHOLD TUNED ON THE TRAINING FOLD.

Method	F_1	Accuracy	Time
Least squares (normal eq.)	0.430 ± 0.006	0.876	55 s
Ridge regression	0.430 ± 0.006	0.875	71 s
MSE gradient descent	0.397 ± 0.004	0.855	166 s
MSE stochastic GD	0.385 ± 0.012	0.851	62 s
Logistic regression (GD)	0.405 ± 0.005	0.866	174 s
Reg. logistic (GD)	0.404 ± 0.005	0.866	165 s
Reg. logistic (Newton)	0.441 ± 0.006	0.877	191 s
Newton, default threshold	0.259 ± 0.003	0.917	196 s

TABLE II
ABLATION OF THE PIPELINE, 3-FOLD CROSS-VALIDATION, REGULARISED LOGISTIC REGRESSION FITTED BY NEWTON'S METHOD.

Configuration	F_1	ΔF_1
Full pipeline	0.440 ± 0.002	—
− quadratic terms	0.439 ± 0.001	-0.000
− one-hot encoding	0.439 ± 0.001	-0.001
− missingness indicators	0.433 ± 0.001	-0.007
− reserved-code masking	0.438 ± 0.002	-0.002
− compound decoding	0.439 ± 0.001	-0.001
− columns $> 50\%$ missing	0.426 ± 0.001	-0.013
No cleaning at all	0.430 ± 0.002	-0.010
+ balanced class weights	0.438 ± 0.002	-0.002

Adding the quadratic features multiplies $\lambda_{\max}$ by 15, so a step size tuned before the feature expansion silently diverges afterwards — a trap we fell into.

B. Newton's method

Gradient descent on this data set is slow because the features have very different curvatures. Since the design matrix has only a few hundred columns, we can afford the exact Newton step

$$w \leftarrow w - H^{-1}\nabla\mathcal{L}(w), \qquad H = \tfrac{1}{N}X^{\top}SX + 2\lambda I,$$

where $S = \operatorname{diag}(\sigma(1-\sigma))$ collects the curvature of each sample. This is one (P, P) linear solve per iteration on top of the same matrix products gradient descent already performs. Newton converges in about ten iterations where 300 gradient steps are still far from the optimum, for a comparable total cost (Table I).

C. The decision threshold

A model trained on a set with $8.8\,\%$ positives puts almost every predicted probability below 0.5, so it flags far too few respondents: with the default threshold our model reaches $F_1 = 0.259$ despite an accuracy of $91.7\,\%$, its highest accuracy of all. We therefore treat the threshold as a hyperparameter and select the value maximising F_1 on out-of-fold predictions. The whole F_1 curve costs a single sort, since after ordering the scores the number of true positives above a threshold is a cumulative sum.

IV. RESULTS AND DISCUSSION

a) The threshold dominates everything.: Comparing Table I with the same model evaluated at the default threshold shows the single largest effect we measured: F_1 falls from 0.441 to 0.259 while accuracy *rises* from 0.877 to 0.917. The two metrics disagree because they answer different questions: optimising the wrong one produces a model that misses most patients in order to protect a majority it was never asked about. Any report on this data set that quotes accuracy is reporting the prevalence of the disease, not the quality of the model.

b) Our careful cleaning was largely wasted.: Removing the reserved-code masking, the compound decoding, the one-hot encoding or the quadratic terms costs at most 0.002 of F_1 each, and a deliberately naive pipeline — every column read as a raw number, median-imputed and standardised — lands 0.010 below the full pipeline, less than the gap between gradient descent and Newton. Two explanations fit: the codes 7 and 9 are numerically far from the valid range 1–5, so a linear model isolates them with one coefficient anyway, and the BRFSS derives dozens of variables from the same raw answers, so corrupting one representation of age leaves several clean ones. We report this negative result because it would have changed how we spent our time.

c) What did matter: concerns *missingness* and *optimisation*, not the content of the answers. Keeping every column, including those missing more than $90\,\%$ of the time, is worth $+0.013$ over dropping everything above $50\,\%$ missingness, and the missingness indicators are worth $+0.007$ — by far the largest contribution of the pipeline. A question asked only to a sub-sample identifies that sub-sample: the pattern of non-answers encodes which survey module a respondent was routed through. Replacing gradient descent by Newton's method is worth $+0.035$ at comparable cost, because 300 gradient steps are nowhere near the optimum. And tuning the threshold is worth $+0.18$.

d) Limitations.: Our model is linear; the quadratic terms are a crude substitute for the interactions a tree ensemble would find and bought only $+0.0004$, while pairwise products of the twenty most correlated features added $+0.0003$ for 145 columns, so we dropped them. Balanced class weights made things slightly worse once the threshold was tuned, as expected: reweighting classes and moving the threshold do the same thing.

V. CONCLUSION

Regularised logistic regression fitted by Newton's method on the encoded BRFSS survey reaches an out-of-fold F_1 of 0.441 at $87.7\,\%$ accuracy, using only NumPy. The ablation ranks our decisions unambiguously: the threshold ($+0.18$) is worth five times the optimiser ($+0.035$), itself worth three times the feature pipeline ($+0.010$). On an imbalanced problem, what you optimise matters more than how.